

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NEHEMIA PANDU INDRAGIRI

109082500019

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Sorting adalah suatu proses menyusun kembali sekumpulan objek atau data ke dalam suatu urutan tertentu berdasarkan aturan tertentu.

Selection Sort adalah algoritma pengurutan yang beroperasi dengan ide dasar mencari nilai ekstrim (nilai terkecil untuk ascending atau terbesar untuk descending) pada sekumpulan data yang belum terurut, kemudian meletakkan nilai tersebut pada posisi yang seharusnya.

Insertion Sort adalah algoritma pengurutan yang bekerja dengan cara mengambil elemen data satu per satu, kemudian menyisipkan elemen tersebut ke posisi yang tepat pada bagian array yang sudah diurutkan.

B. Guided

1. Sorting

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int

    for i = 0; i < len(arr) - 1; i++ { //perulangan
        luar
            idx_min = i;
            for j = i + 1; j < len(arr); j++ {
                //perulangan dalam
                if arr[j] < arr[idx_min] {
                    //kondisional
                    idx_min = j
                }
            }

            if idx_min != i { //swap
                arr[i], arr[idx_min] = arr[idx_min], arr[i]
            }
        }
    }

    func main() {
        var arrAngka [8]int

        for i := 0; i < len(arrAngka); i++ {
            fmt.Printf("Masukkan data angka indeks ke-
%d: ", i)
            fmt.Scan(&arrAngka[i])
        }
    }
}
```

```

        fmt.Println()
        fmt.Println("=== SEBELUM SORTING ===")
        for i := 0; i < len(arrAngka); i++ {
            fmt.Print(arrAngka[i], " ")
        }

        fmt.Println()
        SelectionSortArray(&arrAngka)
        fmt.Println("=== SETELAH SORTING ===")
        for i := 0; i < len(arrAngka); i++ {
            fmt.Print(arrAngka[i], " ")
        }
    }
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\A
Masukkan data angka indeks ke-0: 1
Masukkan data angka indeks ke-1: 3
Masukkan data angka indeks ke-2: 4
Masukkan data angka indeks ke-3: 5
Masukkan data angka indeks ke-4: 6
Masukkan data angka indeks ke-5: 2
Masukkan data angka indeks ke-6: 4
Masukkan data angka indeks ke-7: 6

=== SEBELUM SORTING ===
1 3 4 5 6 2 4 6
=== SETELAH SORTING ===
1 2 3 4 4 5 6 6

```

2. Sorting

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int

```

```

        for i = 0; i < len(sMHS) - 1; i++ { //loop luar
            idx_min = i
            for j = i + 1; j < len(sMHS); j++ { //loop dalam
                mencari nilai ekstrim
                if sMHS[j].IPK < sMHS[idx_min].IPK {
                    //sorting terkecil ke terbesar
                    idx_min = j
                }
            }
            if idx_min != i {
                sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
            }
        }
    }

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n", i)

        fmt.Print("Nama: ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM: ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK: ")
        fmt.Scan(&structMHS[i].IPK)
    }

    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }

    fmt.Println("=====")
    fmt.Println()
    SelectionSortArray(&structMHS)
    fmt.Println("=== SETELAH SORTING ===")

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
}

```

```

        fmt.Println("=====")
        fmt.Println()
    }

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go
--- Masukkan Data Mahasiswa ke-0 ---
Nama: pandu
NIM: 12
IPK: 4
--- Masukkan Data Mahasiswa ke-1 ---
Nama: mia
NIM: 123
IPK: 3.7
--- Masukkan Data Mahasiswa ke-2 ---
Nama: indra
NIM: 345
IPK: 3.5
--- Masukkan Data Mahasiswa ke-3 ---
Nama: giri
NIM: 5567
IPK: 3.6
--- Masukkan Data Mahasiswa ke-4 ---
Nama: nehemia
NIM: 789
IPK: 3.9

=== SEBELUM SORTING ===
--- Data Mahasiswa ke-0 ---
Nama: pandu
NIM: 12
IPK: 4.00
--- Data Mahasiswa ke-1 ---
Nama: mia
NIM: 123
IPK: 3.70
--- Data Mahasiswa ke-2 ---
Nama: indra
NIM: 345
IPK: 3.50
--- Data Mahasiswa ke-3 ---
Nama: giri
NIM: 5567
IPK: 3.60
--- Data Mahasiswa ke-4 ---
Nama: nehemia
NIM: 789
IPK: 3.90
=====

=== SETELAH SORTING ===
--- Data Mahasiswa ke-0 ---
Nama: indra
NIM: 345
IPK: 3.50
--- Data Mahasiswa ke-1 ---
Nama: giri
NIM: 5567
IPK: 3.60
--- Data Mahasiswa ke-2 ---
Nama: mia
NIM: 123
IPK: 3.70
--- Data Mahasiswa ke-3 ---
Nama: nehemia
NIM: 789
IPK: 3.90
--- Data Mahasiswa ke-4 ---
Nama: pandu
NIM: 12
IPK: 4.00
=====

```

3. Sorting

```

package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n - 1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j - 1] {
            T[j] = T[j - 1]
            j--
        }
        T[j] = temp
    }
}

func main() {

```

```

var data arrInt
var n, i int

fmt.Print("Masukkan jumlah data: ")
fmt.Scan(&n)
fmt.Println("Masukkan data: ")

for i = 0; i < n; i++ {
    fmt.Scan(&data[i])
}
fmt.Println("\nData sebelum sorting: ")

for i = 0; i < n; i++ {
    fmt.Print(data[i], " ")
}

insertionSort(&data, n)
fmt.Println("\n\nData setelah sorting (Descending): ")

for i = 0; i < n; i++ {
    fmt.Print(data[i], " ")
}
fmt.Println()
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Praktikum 2\
Masukkan jumlah data: 3
Masukkan data:
16 2 7

Data sebelum sorting:
16 2 7

Data setelah sorting (Descending):
16 7 2
PS D:\Semester 2\Alpro Praktikum 2>

```

4. Sorting

```

package main

import "fmt"

type mahasiswa struct {
    nama string
}

```

```

        nim string
        ipk float64
    }

    type arrMhs [100]mahasiswa

    func insertionSortStruct(T *arrMhs, n int) {
        var temp mahasiswa
        var i, j int
        for i = 1; i < n; i++ {
            temp = T[i]
            j = i - 1
            for j >= 0 && T[j].nama < temp.nama {
                T[j + 1] = T[j]
                j--
            }
            T[j + 1] = temp
        }
    }

    func main() {
        var data arrMhs
        var n, i int

        fmt.Print("Masukkan jumlah mahasiswa: ")
        fmt.Scan(&n)

        for i = 0; i < n; i++ {
            fmt.Println("\nData mahasiswa ke-", i
+ 1)

            fmt.Print("Nama: ")
            fmt.Scan(&data[i].nama)
            fmt.Print("NIM: ")
            fmt.Scan(&data[i].nim)
            fmt.Print("IPK: ")
            fmt.Scan(&data[i].ipk)
        }

        fmt.Println("\nData sebelum sorting: ")

        for i = 0; i < n; i++ {
            fmt.Println(data[i].nama, data[i].nim,
data[i].ipk)
        }

        insertionSortStruct(&data, n)
    }

```

```
        fmt.Println("\nData setelah sorting  
(Descendingberdasarkan Nama): ")  
  
        for i = 0; i < n; i++ {  
            fmt.Println(data[i].nama, data[i].nim,  
data[i].ipk)  
        }  
    }
```

Output:

```
PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester  
Masukkan jumlah mahasiswa: 3
```

```
Data mahasiswa ke- 1
```

```
Nama: pandu
```

```
NIM: 123
```

```
IPK: 4
```

```
Data mahasiswa ke- 2
```

```
Nama: nehemia
```

```
NIM: 456
```

```
IPK: 3.7
```

```
Data mahasiswa ke- 3
```

```
Nama: indra
```

```
NIM: 789
```

```
IPK: 3.4
```

```
Data sebelum sorting:
```

```
pandu 123 4
```

```
nehemia 456 3.7
```

```
indra 789 3.4
```

```
Data setelah sorting (Descendingberdasarkan Nama):
```

```
pandu 123 4
```

```
nehemia 456 3.7
```

```
indra 789 3.4
```

```
PS D:\Semester 2\Alpro Praktikum 2> |
```

C. Unguided

1. Soal Unguided 1

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i+1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        temp := arr[idxMin]
        arr[idxMin] = arr[i]
        arr[i] = temp
    }
}
```

```

    }

    func main() {
        var n int
        fmt.Scan(&n)

        for i := 0; i < n; i++ {
            var m int
            fmt.Scan(&m)
            var rumah []int
            for j := 0; j < m; j++ {
                var nomor int
                fmt.Scan(&nomor)
                rumah = append(rumah, nomor)
            }
            selectionSort(rumah, m)
            for j := 0; j < m; j++ {
                if j > 0 {
                    fmt.Print(" ")
                }
                fmt.Print(rumah[j])
            }
            fmt.Println()
        }
    }
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Praktikum 2\main.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS D:\Semester 2\Alpro Praktikum 2> 

```

Penjelasan :

Program diatas merupakan program yang dibuat untuk mengurutkan daftar nomor rumah dalam beberapa kelompok data (test case) secara menaik (ascending) menggunakan algoritma Selection Sort. Program ini bekerja dengan cara menerima input jumlah kelompok data , lalu pada setiap kelompoknya, program membaca jumlah nomor rumah , menyimpan seluruh nomor tersebut

ke dalam sebuah slice (rumah), mengurutkannya dari nilai terkecil hingga terbesar dengan mencari elemen minimum secara berulang, dan langsung mencetak hasil nomor rumah yang telah terurut rapi dipisahkan oleh spasi untuk setiap kelompoknya.

2. Soal Unguided 2

- 2) Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i+1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }

        temp := arr[idxMin]
        arr[idxMin] = arr[i]
        arr[i] = temp
    }
}

func selectionSortDesc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMax := i
        for j := i+1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }

        temp := arr[idxMax]
        arr[idxMax] = arr[i]
        arr[i] = temp
    }
}

func main() {
    var n int
    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)
        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }
    }
}
```

```

    }
    selectionSortAsc(ganjil, len(ganjil))
    first := true
    for j := 0; j < len(ganjil); j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(ganjil[j])
        first = false
    }

    for j := 0; j < len(genap); j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(genap[j])
        first = false
    }
    fmt.Println()
}
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Prak
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\Semester 2\Alpro Praktikum 2>

```

Penjelasan :

Program diatas merupakan program yang dibuat untuk memisahkan dan mengurutkan nomor rumah berdasarkan bilangan ganjil dan genap dalam beberapa kelompok data (test case). Program ini bekerja dengan cara menerima input jumlah kelompok data n dan jumlah nomor rumah m, lalu memisahkan nomor-nomor tersebut ke dalam dua slice berbeda: nomor ganjil dimasukkan ke ganjil dan nomor genap ke genap. Selanjutnya, hanya nomor ganjil yang diurutkan secara menaik (ascending) menggunakan fungsi selectionSortAsc, sedangkan nomor genap dibiarkan sesuai urutan aslinya karena fungsi selectionSortDesc tidak pernah dipanggil di dalam main. Terakhir, program

mencetak seluruh nomor ganjil yang telah terurut diikuti oleh nomor genap dalam satu baris yang dipisahkan oleh spasi untuk setiap kelompok data.

3. Soal Unguided 3

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 17 19 37 -5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main

import "fmt"

func insertionSort(arr []int, n int) {
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j = j - 1
        }
        arr[j] = temp
    }
}

func main() {
    var arr []int
    var input int
    for {
        fmt.Scan(&input)
        if input < 0 {
```

```

        break
    }
    arr = append(arr, input)
}

n := len(arr)
if n > 0 {
    insertionSort(arr, n)
}

for i := 0; i < n; i++ {
    if i > 0 {
        fmt.Print(" ")
    }
    fmt.Print(arr[i])
}

fmt.Println()
if n < 2 {
    fmt.Println("Data berjarak tidak tetap")
} else {
    jarak := arr[1] - arr[0]
    tetap := true

    for i := 1; i < n-1; i++ {
        if arr[i+1]-arr[i] != jarak {
            tetap = false
            break
        }
    }

    if tetap {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}
}

```

Output :

```
PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpr  
4 40 14 8 26 1 38 2 32 -31  
1 2 4 8 14 26 32 38 40  
Data berjarak tidak tetap  
PS D:\Semester 2\Alpro Praktikum 2> █
```

Penjelasan :

Program diatas merupakan program yang dibuat untuk mengurutkan sekumpulan data angka non-negatif secara menaik (ascending) sekaligus memeriksa apakah selisih antar angka yang telah terurut tersebut bersifat konstan atau tidak. Program ini bekerja dengan cara membaca input angka secara terus-menerus di dalam perulangan (loop) hingga mendeteksi angka negatif sebagai penanda berhenti, lalu mengurutkan kumpulan angka tersebut menggunakan algoritma Insertion Sort. Setelah seluruh angka diurutkan dan dicetak dalam satu baris, program akan menghitung selisih (jarak) antara elemen pertama dan kedua, kemudian memeriksa setiap elemen berikutnya untuk memastikan apakah seluruh data memiliki selisih yang sama (tetap) sehingga program dapat menampilkan pesan nilai jarak tersebut atau menampilkan pesan "Data berjarak tidak tetap" jika selisihnya berbeda atau jumlah data kurang dari dua.

4. Soal Unguided 4

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit, &pustaka[i].eksemplar,
            &pustaka[i].tahun, &pustaka[i].rating)
    }
}
```

```

    }

    func CetakTerfavorit(pustaka DaftarBuku, n int) {
        if n == 0 {
            return
        }
        maxRating := pustaka[0].rating
        idx := 0

        for i := 1; i < n; i++ {
            if pustaka[i].rating > maxRating {
                maxRating = pustaka[i].rating
                idx = i
            }
        }

        fmt.Printf("%s %s %s %d\n", pustaka[idx].judul,
            pustaka[idx].penulis, pustaka[idx].penerbit,
            pustaka[idx].tahun)
    }

    func UrutBuku(pustaka *DaftarBuku, n int) {
        for i := 1; i < n; i++ {
            temp := pustaka[i]
            j := i
            for j > 0 && temp.rating > pustaka[j-1].rating {
                pustaka[j] = pustaka[j-1]
                j--
            }
            pustaka[j] = temp
        }
    }

    func Cetak5Terbaru(pustaka DaftarBuku, n int) {
        limit := 5
        if n < 5 {
            limit = n
        }

        for i := 0; i < limit; i++ {
            fmt.Println(pustaka[i].judul)
        }
    }

    func CariBuku(pustaka DaftarBuku, n int, r int) {
        kiri := 0
        kanan := n - 1
        found := false
        idx := -1
        for kiri <= kanan && !found {

```

```

        tengah := (kiri + kanan) / 2
        if pustaka[tengah].rating == r {
            found = true
            idx = tengah
        } else if r > pustaka[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    if found {
        fmt.Printf("%s %s %s %d %d %d\n",
pustaka[idx].judul,
        pustaka[idx].penulis, pustaka[idx].penerbit,
pustaka[idx].tahun,
        pustaka[idx].eksemplar, pustaka[idx].rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating
seperti itu")
    }
}

func main() {
    var pustaka DaftarBuku
    var n, r int

    DaftarkanBuku(&pustaka, &n)
    CetakTerfavorit(pustaka, n)
    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)
    fmt.Scan(&r)
    CariBuku(pustaka, n, r)
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Praktikum 2
6
B01 Algoritma_Dasar Budi Informatika Press 10 2020 85
B02 Pemrograman Go Andi Telkom Press 5 2021 95
B03 Struktur_Data Citra IT_Books 15 2019 70
B04 Basis Data Doni SQL Pub 8 2022 88
Struktur_Data Citra IT_Books 2019
Struktur_Data
Algoritma_Dasar
10
Pemrograman
Press
10 2020 85 0 0 0
PS D:\Semester 2\Alpro Praktikum 2>

```

Penjelasan :

Program diatas merupakan program yang dibuat untuk mengelola sistem pendataan dan pencarian buku perpustakaan berbasis struktur data array (DaftarBuku) dari tipe data bentukan struct (Buku). Program ini bekerja dengan cara membaca data buku sejumlah n yang meliputi ID, judul, penulis, penerbit, eksemplar, tahun, serta rating, lalu langsung mencari dan mencetak identitas buku dengan rating tertinggi melalui fungsi CetakTerfavorit. Selanjutnya, program mengurutkan seluruh daftar buku secara menurun (descending) berdasarkan ratingnya menggunakan algoritma Insertion Sort lewat fungsi UrutBuku, mencetak hingga 5 judul buku teratas dari hasil pengurutan tersebut melalui fungsi Cetak5Terbaru, dan diakhiri dengan melakukan pencarian data buku tertentu berdasarkan rating yang diinputkan pengguna memanfaatkan algoritma pencarian biner (Binary Search) pada fungsi CariBuku.

D. Kesimpulan

Praktikum ini membahas materi pengurutan data menggunakan algoritma Selection Sort dan Insertion Sort, di mana Selection Sort bekerja dengan cara memindai seluruh elemen untuk mencari nilai ekstrem (minimum atau maksimum) lalu menukarnya ke posisi yang tepat, sementara Insertion Sort bekerja mirip seperti mengurutkan kartu nama dengan cara menyisipkan setiap elemen baru ke dalam sub-array yang sudah terurut sebelumnya. Kedua metode ini dapat

diimplementasikan secara menaik (ascending) maupun menurun (descending), dan proses pengurutan ini menjadi tahapan krusial karena struktur data yang telah terurut rapi merupakan syarat mutlak agar algoritma pencarian biner (Binary Search) dapat berfungsi dengan tepat dan efisien.